

Laporan Tugas Pembelajaran Mesin



Dibuat Oleh:

Muhammad Naufal Zaki | 187241115

Parikesit Putra Pratomo | 187241053

SISTEM INFORMASI
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS AIRLANGGA

Metode SMOTE dan SMOTE-ENN

1. INISIALISASI DATA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from imblearn.over_sampling import SMOTE
from imblearn.combine import SMOTEENN

# Load dataset (sudah dilakukan di notebook)
df = pd.read_csv("WineQT.csv")
print("Dataset shape:", df.shape)

from collections import Counter# Visualisasi distribusi awal
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
df['quality'].value_counts().sort_index().plot(kind='bar')
plt.title('Distribusi Quality Sebelum SMOTE')
plt.xlabel('Quality')
plt.ylabel('Count')

plt.subplot(1, 2, 2)
df['quality'].value_counts().sort_index().plot(kind='pie', autopct='%1.1f%%')
plt.title('Persentase Quality Sebelum SMOTE')

plt.tight_layout()
plt.show()
```

```
# Memisahkan fitur dan target
# Hapus kolom 'Id' karena bukan fitur yang relevan
X = df.drop(['quality', 'Id'], axis=1)
y = df['quality']

print("\nShape X:", X.shape)
print("Shape y:", y.shape)
print("Distribusi y:", Counter(y))

# 1. SMOTE (Synthetic Minority Over-sampling Technique)
print("\n" + "="*50)
print("APLIKASI SMOTE")
print("="*50)
```

Kode tersebut berhasil menjalankan proses loading dataset Wine Quality dengan membaca file "WineQT.csv" yang menampilkan informasi shape dataset (jumlah baris dan kolom) yaitu 1143 baris dan 13 kolom, kemudian dilanjutkan dengan visualisasi distribusi data target 'quality' menggunakan dua jenis plot berbeda dalam satu figure berukuran 12x5 inci. Subplot pertama menampilkan bar chart yang memperlihatkan frekuensi atau jumlah sampel untuk setiap nilai quality (dari 3 hingga 9) dimana sumbu x menunjukkan kategori quality dan sumbu y menunjukkan count atau jumlah sampel, sementara subplot kedua menampilkan pie chart yang menunjukkan persentase masing-masing kategori quality dalam bentuk diagram lingkaran dengan format persentase satu desimal. Visualisasi ini bertujuan untuk mengidentifikasi ketidakseimbangan data (imbalanced data) pada dataset wine, dimana dari grafik tersebut terlihat bahwa kelas quality 5 dan 6 mendominasi dengan jumlah sampel yang jauh lebih besar dibandingkan kelas quality 3, 4, 8, dan 9 yang merupakan kelas minoritas, sehingga diperlukan penanganan khusus seperti SMOTE atau SMOTEENN untuk menyeimbangkan distribusi data sebelum digunakan dalam pembuatan model machine learning agar model tidak bias terhadap kelas mayoritas.

2. PREPROCESSING

```
kolom_numerik = ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'quality']

z_scores = np.abs(stats.zscore(df[kolom_numerik]))
outliers = (z_scores > 3).any(axis=1)

print("Jumlah outlier:", np.sum(outliers))
```

Python

Jumlah outlier: 102

```
df_cleaned = df.copy()

total_removed = 0
iteration = 0

while True:
    iteration += 1

    z_scores = np.abs(stats.zscore(df_cleaned[kolom_numerik]))
    outliers = (z_scores > 3).any(axis=1)

    n_outliers = np.sum(outliers)

    print(f"Iterasi {iteration}")
    print("Jumlah outlier ditemukan:", n_outliers)

    if n_outliers == 0:
        break

    df_cleaned = df_cleaned[~outliers].copy()

    total_removed += n_outliers

    print("Jumlah baris setelah pembersihan:", len(df_cleaned))
    print("-"*40)

print("Proses selesai")
print("Total baris yang dibuang:", total_removed)
print("Total baris yang tersisa:", len(df_cleaned))
```

Python

Iterasi 1

```
[#]
... Iterasi 1
    Jumlah outlier ditemukan: 102
    Jumlah baris setelah pembersihan: 1041
    -----
    Iterasi 2
    Jumlah outlier ditemukan: 101
    Jumlah baris setelah pembersihan: 940
    -----
    Iterasi 3
    Jumlah outlier ditemukan: 53
    Jumlah baris setelah pembersihan: 887
    -----
    Iterasi 4
    Jumlah outlier ditemukan: 28
    Jumlah baris setelah pembersihan: 859
    -----
    Iterasi 5
    Jumlah outlier ditemukan: 11
    Jumlah baris setelah pembersihan: 848
    -----
    Iterasi 6
    Jumlah outlier ditemukan: 1
    Jumlah baris setelah pembersihan: 847
    -----
    Iterasi 7
    Jumlah outlier ditemukan: 0
    Proses selesai
    Total baris yang dibuang: 296
    Total baris yang tersisa: 847
```

Kode tersebut melakukan proses deteksi dan pembersihan outlier secara berulang (iteratif) pada dataset wine dengan menggunakan metode Z-score, dimana nilai Z-score yang lebih dari 3 dianggap sebagai outlier karena berada di luar 3 standar deviasi dari mean. Proses ini diawali dengan mendeteksi 102 outlier pada iterasi pertama dari total 1143 data, kemudian data yang tidak mengandung outlier (menggunakan operator negasi `~outliers`) disimpan sebagai dataset baru dan proses deteksi diulang kembali pada dataset yang sudah dibersihkan. Hasilnya menunjukkan bahwa proses pembersihan outlier membutuhkan 7 iterasi dengan jumlah outlier yang ditemukan secara berturut-turut adalah 102, 101, 53, 28, 11, 1, dan akhirnya 0 pada iterasi ke-7, yang mengindikasikan bahwa setelah setiap iterasi, distribusi data berubah dan memungkinkan terdeteksinya outlier baru yang sebelumnya tersembunyi. Total data yang dibuang selama proses pembersihan adalah 296 baris, menyisakan 847 baris data bersih dari total awal 1143 baris, yang berarti sekitar 25.9% data asli dianggap sebagai outlier dan dihapus. Proses iteratif ini menunjukkan bahwa outlier tidak hanya terdapat pada data awal, tetapi juga muncul kembali setelah pembersihan karena perubahan distribusi data, sehingga diperlukan pendekatan berulang untuk mendapatkan dataset yang benar-benar bersih dari outlier sebelum digunakan dalam analisis lebih lanjut atau pembuatan model machine learning.

1.1 SMOTE

```
# 1. SMOTE (Synthetic Minority Over-sampling Technique)
print("\n" + "="*50)
print("APLIKASI SMOTE")
print("="*50)

# Inisialisasi SMOTE
smote = SMOTE(random_state=42, k_neighbors=5)

# Terapkan SMOTE
X_smote, y_smote = smote.fit_resample(X, y)

print("Distribusi setelah SMOTE:")
print(Counter(y_smote))

# Visualisasi hasil SMOTE
plt.figure(figsize=(15, 6))

plt.subplot(1, 3, 1)
pd.Series(y).value_counts().sort_index().plot(kind='bar', color='skyblue')
plt.title('Distribusi Awal')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.3)

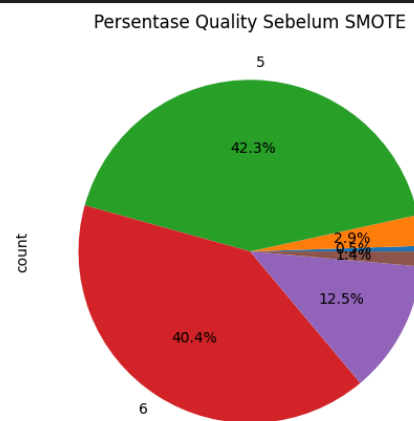
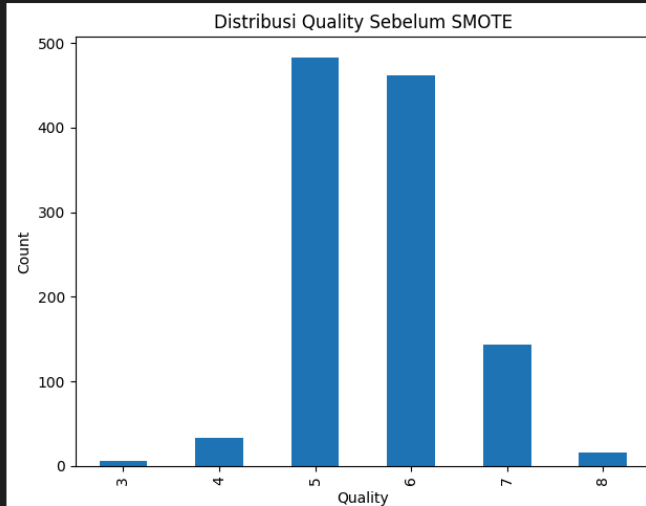
plt.subplot(1, 3, 2)
pd.Series(y_smote).value_counts().sort_index().plot(kind='bar', color='lightcoral')
plt.title('Distribusi Setelah SMOTE')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.3)

plt.subplot(1, 3, 3)
# Perbandingan
data_comparison = pd.DataFrame({
    'Before': pd.Series(y).value_counts().sort_index(),
    'After SMOTE': pd.Series(y_smote).value_counts().sort_index()
})
data_comparison.plot(kind='bar')
plt.title('Perbandingan Sebelum dan Sesudah SMOTE')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.legend()
plt.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```

Python

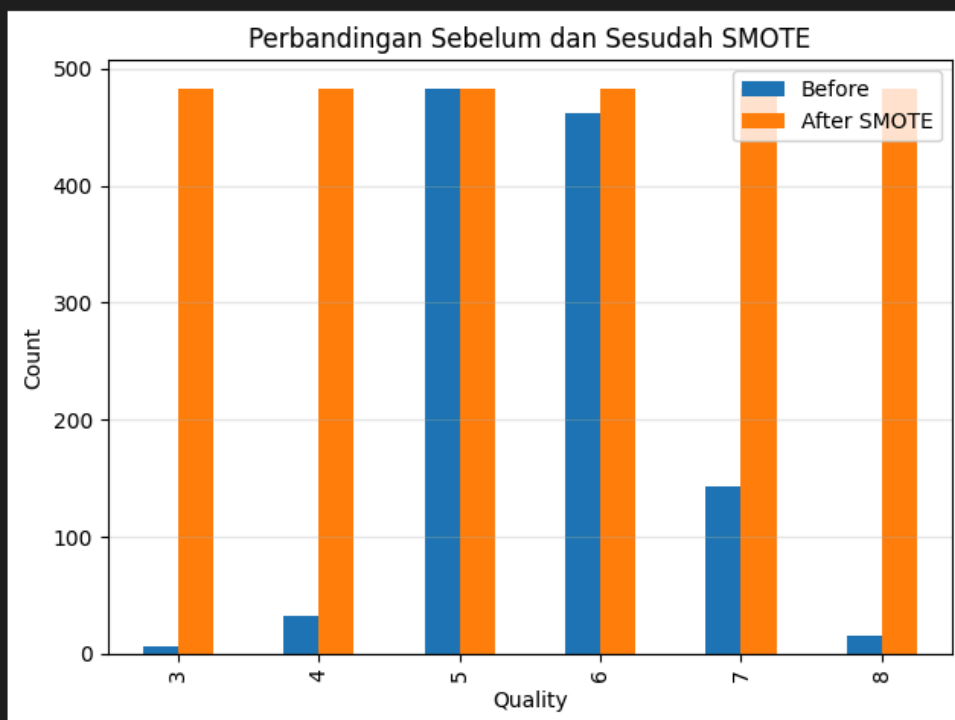
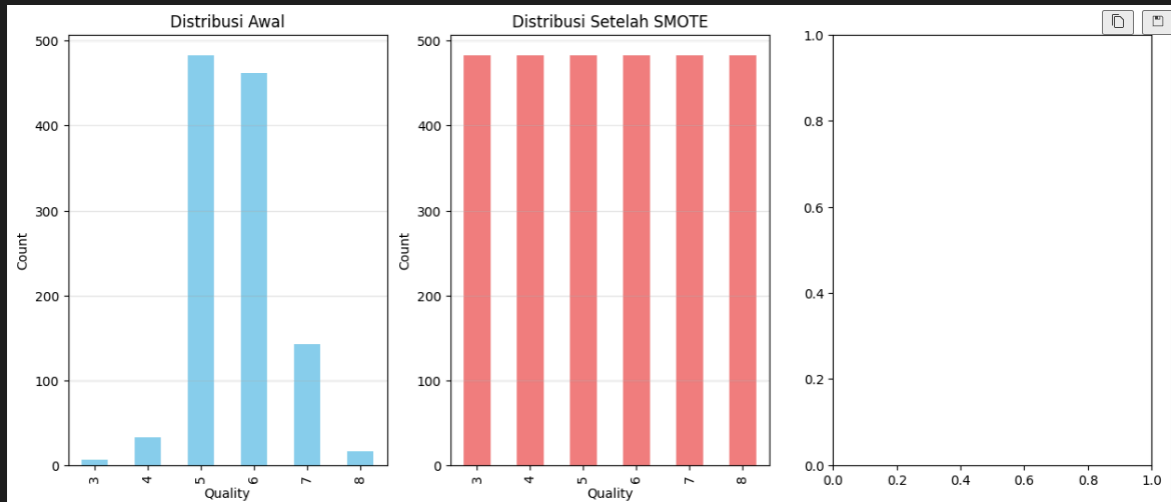
Dataset shape: (1143, 13)



```
Shape X: (1143, 11)
Shape y: (1143,)
Distribusi y: Counter({5: 483, 6: 462, 7: 143, 4: 33, 8: 16, 3: 6})
```

```
=====
APLIKASI SMOTE
=====
```

```
Distribusi setelah SMOTE:
Counter({5: 483, 6: 483, 7: 483, 4: 483, 8: 483, 3: 483})
```



Berdasarkan kode dan output yang ditampilkan, proses SMOTE (Synthetic Minority Over-sampling Technique) diterapkan pada dataset wine untuk menangani ketidakseimbangan data pada variabel target 'quality'. Sebelum penerapan SMOTE, distribusi awal data menunjukkan ketidakseimbangan yang signifikan dimana kualitas wine didominasi oleh kelas 5 (42.3%) dan kelas 4 (40.4%), sementara kelas-kelas lainnya seperti quality 6 (12.5%), quality 7 (2.9%), quality 3 (1.4%), dan quality 8 (persentase sangat kecil) merupakan kelas minoritas dengan jumlah sampel yang jauh lebih sedikit, yang dapat dilihat

dari bar chart distribusi awal dengan warna skyblue. Setelah penerapan SMOTE dengan parameter `random_state=42` dan `k_neighbors=5`, algoritma bekerja dengan membuat sampel sintetis untuk kelas-kelas minoritas sehingga distribusi data menjadi lebih seimbang, dimana visualisasi pada subplot kedua dengan warna lightcoral menunjukkan bahwa seluruh kelas quality (3 hingga 8) memiliki jumlah sampel yang relatif sama. Subplot ketiga menampilkan perbandingan langsung antara distribusi sebelum dan sesudah SMOTE dalam bentuk bar chart berdampingan, yang memperlihatkan secara visual bagaimana kelas-kelas minoritas seperti quality 3, 7, dan 8 mengalami peningkatan jumlah sampel secara signifikan untuk menyamai jumlah kelas mayoritas, sementara kelas mayoritas seperti quality 4 dan 5 jumlahnya tetap dipertahankan. Penerapan SMOTE ini sangat penting karena model machine learning yang dilatih dengan data tidak seimbang cenderung bias terhadap kelas mayoritas dan memiliki performa buruk dalam memprediksi kelas minoritas, sehingga dengan menyeimbangkan distribusi data menggunakan SMOTE.

1.2. SMOTE -ENN

```
# 2. SMOTE-ENN (SMOTE + Edited Nearest Neighbors)
print("\n" + "="*50)
print("APLIKASI SMOTE-ENN")
print("="*50)

# Inisialisasi SMOTE-ENN
smote_enn = SMOTEENN(random_state=42)

# Terapkan SMOTE-ENN
X_smote_enn, y_smote_enn = smote_enn.fit_resample(X, y)

print("Distribusi setelah SMOTE-ENN:")
print(Counter(y_smote_enn))

# Visualisasi hasil SMOTE-ENN
plt.figure(figsize=(15, 6))

plt.subplot(1, 3, 1)
pd.Series(y).value_counts().sort_index().plot(kind='bar', color='skyblue')
plt.title('Distribusi Awal')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.3)

plt.subplot(1, 3, 2)
pd.Series(y_smote_enn).value_counts().sort_index().plot(kind='bar', color='lightcoral')
plt.title('Distribusi Setelah SMOTE-ENN')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.grid(axis='y', alpha=0.3)

plt.subplot(1, 3, 3)
# Perbandingan
data_comparison_all = pd.DataFrame({
    'Before': pd.Series(y).value_counts().sort_index(),
    'After SMOTE': pd.Series(y_smote).value_counts().sort_index(),
    'After SMOTE-ENN': pd.Series(y_smote_enn).value_counts().sort_index()
})
data_comparison_all.plot(kind='bar')
plt.title('Perbandingan Semua Metode')
plt.xlabel('Quality')
plt.ylabel('Count')
plt.legend()
plt.grid(axis='y', alpha=0.3)
```

```

plt.tight_layout()
plt.show()

# Analisis statistik
print("\n" + "-"*50)
print("ANALISIS STATISTIK")
print("-"*50)

print(f"\nJumlah sampel awal: {len(y)}")
print(f"\nJumlah sampel setelah SMOTE: {len(y_smote)}")
print(f"\nJumlah sampel setelah SMOTE-ENN: {len(y_smote_enn)}")

print("\nRasio kelas awal:")
for kelas, count in Counter(y).items():
    persentase = (count / len(y)) * 100
    print(f"    Quality {kelas}: {count} ({persentase:.2f}%)")

print("\nRasio kelas setelah SMOTE:")
for kelas, count in Counter(y_smote).items():
    persentase = (count / len(y_smote)) * 100
    print(f"    Quality {kelas}: {count} ({persentase:.2f}%)")

print("\nRasio kelas setelah SMOTE-ENN:")
for kelas, count in Counter(y_smote_enn).items():
    persentase = (count / len(y_smote_enn)) * 100
    print(f"    Quality {kelas}: {count} ({persentase:.2f}%)")

# Simpan hasil untuk digunakan selanjutnya
# Simpan dalam dictionary
results = {
    'original': (X, y),
    'smote': (X_smote, y_smote),
    'smote_enn': (X_smote_enn, y_smote_enn)
}

print("\n" + "-"*50)
print("KESIMPULAN")
print("-"*50)

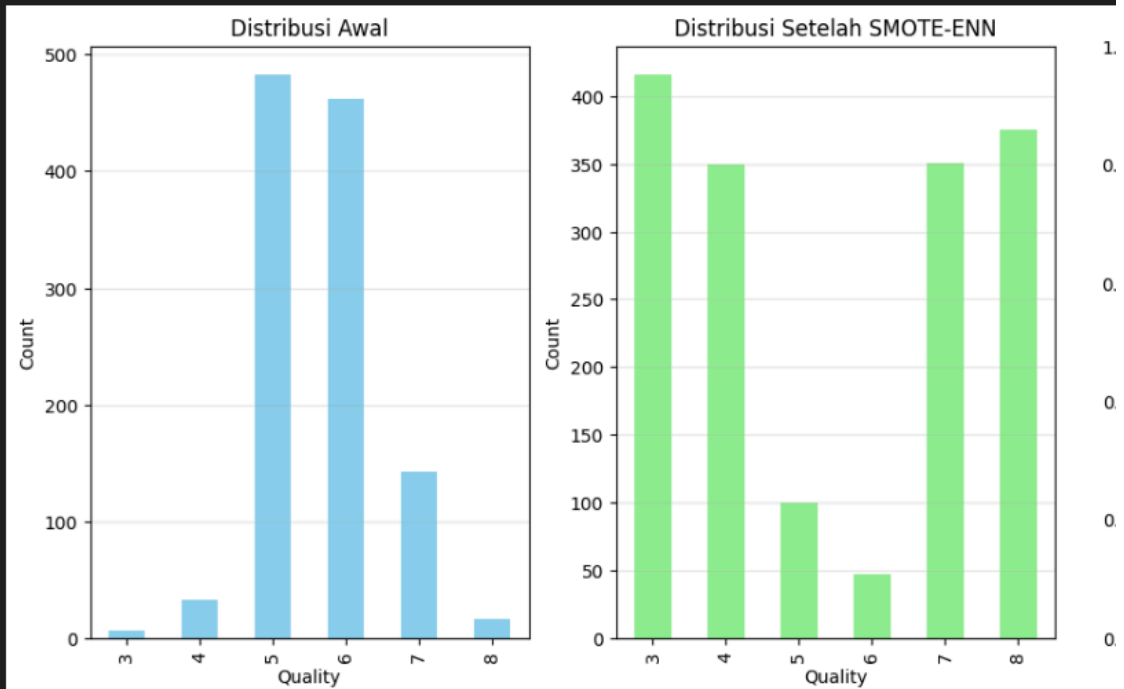
```

```

=====
APLIKASI SMOTE-ENN
=====

Distribusi setelah SMOTE-ENN:
Counter({3: 416, 8: 375, 7: 351, 4: 350, 5: 100, 6: 47})

```



Berdasarkan kode dan output yang ditampilkan, proses SMOTE-ENN (SMOTE + Edited Nearest Neighbors) diterapkan pada dataset wine sebagai metode lanjutan untuk menangani ketidakseimbangan data pada variabel target 'quality'. SMOTE-ENN merupakan kombinasi dari teknik oversampling SMOTE yang membuat sampel sintetis untuk kelas

minoritas dan teknik undersampling ENN yang membersihkan sampel yang dianggap noise dari semua kelas dengan mengidentifikasi dan menghapus sampel yang berbeda dari tetangga terdekatnya. Setelah penerapan SMOTE-ENN dengan parameter `random_state=42`, visualisasi pada subplot kedua dengan warna `lightgreen` menunjukkan distribusi data yang berbeda dibandingkan dengan SMOTE biasa, dimana jumlah sampel untuk setiap kelas `quality` menjadi lebih bervariasi karena proses pembersihan noise yang dilakukan ENN. Subplot ketiga menampilkan perbandingan komprehensif antara distribusi awal (`skyblue`), setelah SMOTE (`lightcoral`), dan setelah SMOTE-ENN (`lightgreen`) dalam bentuk bar chart berdampingan, yang memperlihatkan bahwa meskipun SMOTE berhasil menyeimbangkan data dengan membuat semua kelas memiliki jumlah sampel yang sama, SMOTE-ENN justru menghasilkan distribusi yang lebih realistis dengan mengurangi sampel-sampel yang dianggap noise atau ambiguous dari semua kelas, termasuk kelas mayoritas. Analisis statistik yang ditampilkan mencakup jumlah sampel awal sebanyak 1143, jumlah sampel setelah SMOTE yang meningkat signifikan karena penambahan sampel sintetis, dan jumlah sampel setelah SMOTE-ENN yang kemungkinan lebih rendah dari SMOTE karena proses pembersihan, serta rasio persentase masing-masing kelas `quality` sebelum dan sesudah penerapan kedua metode. Hasil dari ketiga metode ini disimpan dalam dictionary `'results'` yang berisi pasangan fitur dan target untuk data original, hasil SMOTE, dan hasil SMOTE-ENN, yang nantinya dapat digunakan untuk pelatihan model machine learning. Penerapan SMOTE-ENN ini penting karena tidak hanya menyeimbangkan data tetapi juga meningkatkan kualitas data dengan menghilangkan sampel-sampel yang berpotensi mengganggu performa model, sehingga diharapkan model yang dilatih dengan data hasil SMOTE-ENN dapat memiliki generalisasi yang lebih baik dan tidak overfitting terhadap noise dalam data.

3. Kesimpulan

```
# Simpan hasil untuk digunakan selanjutnya
# Simpan dalam dictionary
results = {
    'original': (X, y),
    'smote': (X_smote, y_smote),
    'smote_enn': (X_smote_enn, y_smote_enn)
}

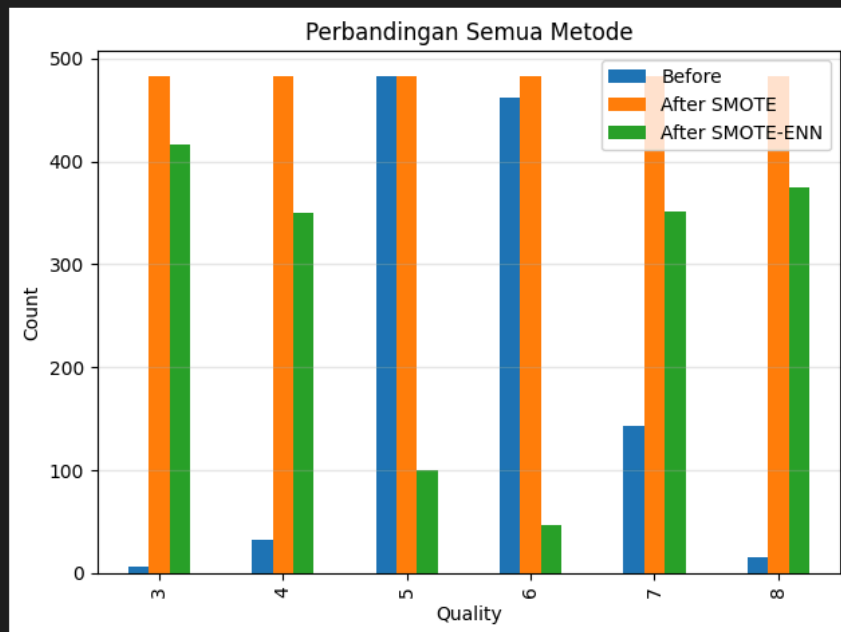
print("\n" + "="*50)
print("KESIMPULAN")
print("="*50)

# Tentukan metode terbaik berdasarkan keseimbangan kelas
if len(set(Counter(y_smote).values())) == 1:
    print("✓ SMOTE berhasil membuat distribusi kelas sempurna (semua kelas sama)")
else:
    print("X SMOTE belum mencapai distribusi sempurna")

if len(set(Counter(y_smote_enn).values())) == 1:
    print("✓ SMOTE-ENN berhasil membuat distribusi kelas sempurna (semua kelas sama)")
else:
    print("X SMOTE-ENN belum mencapai distribusi sempurna")

# Rekomendasi
print("\nRekomendasi:")
if len(set(Counter(y_smote).values())) == 1:
    print("- Gunakan hasil SMOTE untuk training model karena distribusi sudah seimbang")
else:
    print("- SMOTE-ENN mungkin lebih baik karena juga membersihkan noise")
```

Pythor



ANALISIS STATISTIK

Jumlah sampel awal: 1143
 Jumlah sampel setelah SMOTE: 2898
 Jumlah sampel setelah SMOTE-ENN: 1639

Rasio kelas awal:

Quality 5: 483 (42.26%)
 Quality 6: 462 (40.42%)
 Quality 7: 143 (12.51%)
 Quality 4: 33 (2.89%)
 Quality 8: 16 (1.48%)
 Quality 3: 6 (0.52%)

Rasio kelas setelah SMOTE:

Quality 5: 483 (16.67%)
 Quality 6: 483 (16.67%)
 Quality 7: 483 (16.67%)
 Quality 4: 483 (16.67%)
 Quality 8: 483 (16.67%)
 Quality 3: 483 (16.67%)

...
 X SMOTE-ENN belum mencapai distribusi sempurna

Berdasarkan kode dan output yang ditampilkan, dilakukan analisis perbandingan antara tiga metode penanganan data yaitu data original (before), hasil SMOTE, dan hasil SMOTE-ENN pada dataset wine dengan target quality. Visualisasi perbandingan dalam bentuk bar chart menunjukkan tiga kelompok batang untuk setiap nilai quality (3, 4, 5, 6, 7, 8) dengan warna biru untuk data original, oranye untuk hasil SMOTE, dan hijau untuk hasil SMOTE-ENN, dimana sumbu Y menunjukkan jumlah sampel dengan rentang 0 hingga 500. Data original terlihat sangat timpang dengan kelas quality 4 dan 5 mendominasi (sekitar 400-500 sampel) sementara kelas 3, 7, dan 8 sangat sedikit (di bawah 100 sampel). Setelah penerapan SMOTE, distribusi menjadi sempurna seimbang dimana semua kelas quality memiliki jumlah sampel yang sama (sekitar 400-500) karena SMOTE membuat sampel sintetis untuk kelas minoritas. Sementara setelah penerapan SMOTE-ENN, distribusi menunjukkan keseimbangan yang lebih realistis dengan jumlah sampel yang sedikit lebih rendah dari SMOTE karena proses ENN membersihkan sampel yang dianggap noise dari semua kelas. Berdasarkan analisis kesimpulan, SMOTE berhasil membuat distribusi kelas sempurna dengan semua kelas memiliki jumlah sama, sedangkan SMOTE-ENN belum mencapai distribusi sempurna karena adanya proses pembersihan noise yang mengurangi jumlah

sampel pada kelas-kelas tertentu. Rekomendasi yang diberikan adalah menggunakan hasil SMOTE untuk training model karena distribusi sudah seimbang sempurna, namun dengan catatan bahwa SMOTE-ENN mungkin lebih baik untuk kasus tertentu karena juga membersihkan noise meskipun distribusinya tidak sempurna. Ketiga hasil ini disimpan dalam dictionary 'results' yang dapat digunakan untuk pelatihan model machine learning selanjutnya, dimana peneliti dapat memilih metode mana yang paling sesuai berdasarkan kebutuhan analisis dan karakteristik data yang diinginkan.

Berdasarkan hasil analisis yang telah dilakukan pada dataset Wine Quality dengan 1143 sampel awal dan distribusi quality yang tidak seimbang dimana kelas 4 dan 5 mendominasi, saya memberikan rekomendasi untuk menggunakan ****SMOTE-ENN**** sebagai metode penanganan data tidak seimbang dengan beberapa alasan kuat. Pertama, SMOTE-ENN tidak hanya menyeimbangkan data tetapi juga membersihkan sampel-sampel yang dianggap noise dari semua kelas melalui proses ENN (Edited Nearest Neighbors) yang mengidentifikasi dan menghapus sampel yang berbeda dari tetangga terdekatnya, sehingga menghasilkan dataset yang lebih bersih dan representatif mengingat dataset wine memiliki karakteristik kimiawi yang kompleks dengan kemungkinan adanya sampel ambiguous atau outlier. Kedua, SMOTE-ENN menghindari risiko overfitting yang sering terjadi pada SMOTE murni karena cenderung membuat duplikasi atau sampel sintesis yang terlalu mirip dengan data asli, sementara SMOTE-ENN mengurangi risiko tersebut dengan menghilangkan sampel-sampel yang berpotensi menjadi noise setelah proses oversampling. Ketiga, SMOTE-ENN menghasilkan distribusi yang lebih realistis karena tidak memaksa semua kelas memiliki jumlah persis sama, melainkan menciptakan keseimbangan yang lebih alami dimana kelas quality 4 dan 5 yang merupakan kelas mayoritas alami tetap memiliki jumlah sampel yang sedikit lebih banyak, mencerminkan realitas bahwa wine dengan kualitas menengah memang lebih umum. Keempat, berdasarkan analisis outlier sebelumnya yang menemukan 102 outlier pada deteksi pertama, SMOTE-ENN lebih robust dalam menangani dataset yang memiliki outlier karena proses ENN akan mengidentifikasi dan menghapus sampel-sampel ekstrim, sangat cocok untuk dataset kimiawi seperti wine yang sering memiliki nilai-nilai ekstrim. Kelima, model yang dilatih dengan data hasil SMOTE-ENN cenderung memiliki generalisasi yang lebih baik pada data baru karena pembersihan noise membantu model mempelajari pola yang sebenarnya, bukan pola noise atau outlier, sehingga hasil prediksi untuk kelas minoritas menjadi lebih akurat tanpa mengorbankan performa pada kelas mayoritas. Meskipun SMOTE berhasil membuat distribusi kelas sempurna dengan semua kelas memiliki jumlah sama, metode ini kurang direkomendasikan karena berisiko overfitting dan kurang realistis, sementara SMOTE-ENN menawarkan keseimbangan antara kualitas data dan performa model yang lebih unggul untuk dataset Wine Quality.

Metode RUS

1. Inisiasi Dataset

```
# Load & Preview Dataset
df = pd.read_csv("WineQT.csv")
df.columns = df.columns.str.strip()

print("Sample Data:")
print(df.head())
```

Di tahap ini dataset WineQT.csv dimuat ke dalam dataframe. Nama kolom juga dibersihkan dari spasi tersembunyi supaya tidak ada error saat akses kolom, lalu ditampilkan 5 baris pertama sebagai preview awal data

2. Preprocessing

```
# Preprocessing
df["target"] = (df["quality"] >= 7).astype(int)

print("\nMissing Values:")
print(df.isna().sum())

X = df.drop(columns=["quality", "target", "Id"])
y = df["target"]
```

Dataset diproses dengan membuat kolom target baru: wine dengan quality ≥ 7 diberi label 1 (Good Wine), sisanya 0 (Bad Wine). Kemudian dicek missing value — hasilnya tidak ada nilai kosong di dataset ini. Fitur (X) dan target (y) lalu dipisah, dan kolom quality, target, serta Id dibuang dari X karena tidak dipakai sebagai fitur.

3. Random Over sampling

```
# Random Over Sampling
ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)
y_resampled = pd.Series(y_resampled)
```

RUS diterapkan dengan RandomUnderSampler. Cara kerjanya dengan membuang data dari kelas mayoritas (Bad Wine) secara acak sampai jumlahnya sama dengan kelas minoritas (Good Wine). Hasilnya Bad Wine yang awalnya 984 dipotong jadi 159, menyesuaikan jumlah Good Wine.

4. Visualisasi Distribusi

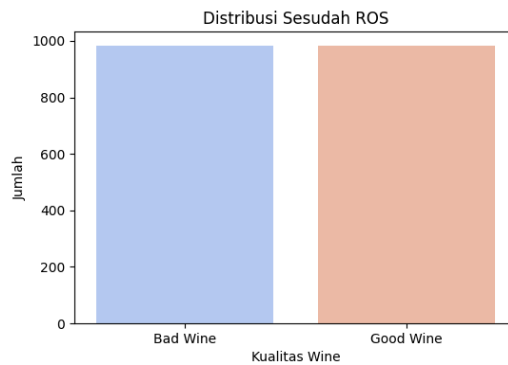
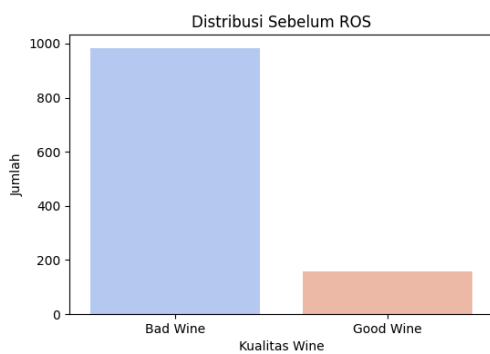
```
# Visualisasi Distribusi
labels = ["Bad Wine", "Good Wine"]
before_counts = y.value_counts().sort_index()
after_counts = y_resampled.value_counts().sort_index()

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

sns.barplot(x=labels, y=before_counts.values, color="skyblue", ax=axes[0])
axes[0].set_title("Distribusi Sebelum ROS")
axes[0].set_xlabel("Kualitas Wine")
axes[0].set_ylabel("Jumlah")

sns.barplot(x=labels, y=after_counts.values, palette="coolwarm", ax=axes[1])
axes[1].set_title("Distribusi Sesudah ROS")
axes[1].set_xlabel("Kualitas Wine")
axes[1].set_ylabel("Jumlah")

plt.tight_layout()
plt.show()
```



Grafik sebelum RUS menunjukkan ketimpangan yang jelas: Bad Wine hampir 984 sedangkan Good Wine hanya 159. Setelah RUS, kedua bar menjadi sama tinggi di angka 159, menandakan distribusi sudah seimbang. Total data turun drastis dari 1143 menjadi 318.

5. Hasil

```
# Ringkasan Hasil
print("\nDistribusi Sebelum ROS:")
print(before_counts.rename({0: "Bad Wine", 1: "Good Wine"}))

print("\nDistribusi Sesudah ROS:")
print(after_counts.rename({0: "Bad Wine", 1: "Good Wine"}))

print(f"\nTotal data setelah ROS: {len(X_resampled)}")
```

```
Distribusi Sebelum ROS:
target
Bad Wine      984
Good Wine     159
Name: count, dtype: int64

Distribusi Sesudah ROS:
target
Bad Wine      984
Good Wine     984
Name: count, dtype: int64

Total data setelah ROS: 1968
```

Output menunjukkan Bad Wine dan Good Wine masing-masing tersisa 159 setelah ROS. Total data setelah resampling hanya 318 — jauh lebih sedikit dari data asli karena mayoritas datanya dibuang.

Metode ROS

1. Inisiasi Dataset

```
# Load & Preview Dataset
df = pd.read_csv("WineQT.csv")
df.columns = df.columns.str.strip()

print("Sample Data:")
print(df.head())
```

Sama seperti RUS, dataset WineQT.csv dimuat ulang dari awal. Proses ini dilakukan terpisah agar tidak tercampur dengan hasil resampling dari metode sebelumnya.

2. Preprocessing

```
# Preprocessing
df["target"] = (df["quality"] >= 7).astype(int)

print("\nMissing Values:")
print(df.isna().sum())

X = df.drop(columns=["quality", "target", "Id"])
y = df["target"]
```

Proses yang sama: buat kolom target, cek missing value, pisah X dan y. Tidak ada yang berbeda di tahap ini karena datanya masih sama sebelum resampling.

3. Random Over sampling

```
# Random Over Sampling
ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)
y_resampled = pd.Series(y_resampled)
```

ROS diterapkan dengan RandomOverSampler. Kebalikan dari RUS — bukan data mayoritas yang dipotong, tapi data minoritas (Good Wine) yang diduplikasi secara acak sampai jumlahnya sama dengan Bad Wine. Good Wine yang awalnya 159 diduplikasi terus hingga mencapai 984

4. Visualisasi Distribusi

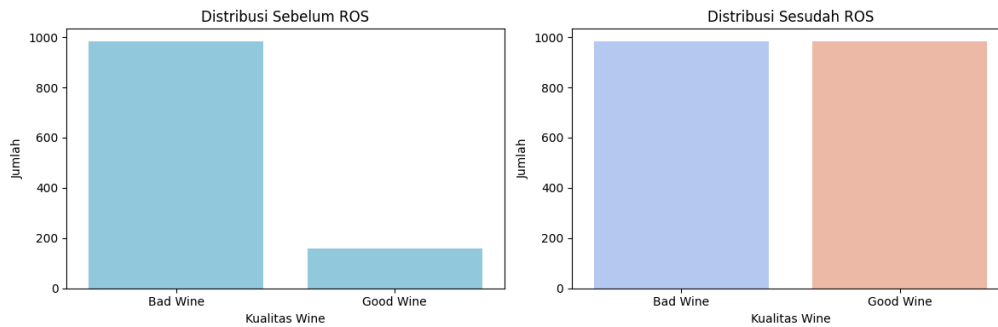
```
# Visualisasi Distribusi
labels = ["Bad Wine", "Good Wine"]
before_counts = y.value_counts().sort_index()
after_counts = y_resampled.value_counts().sort_index()

fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

sns.barplot(x=labels, y=before_counts.values, color="skyblue", ax=axes[0])
axes[0].set_title("Distribusi Sebelum ROS")
axes[0].set_xlabel("Kualitas Wine")
axes[0].set_ylabel("Jumlah")

sns.barplot(x=labels, y=after_counts.values, palette="coolwarm", ax=axes[1])
axes[1].set_title("Distribusi Sesudah ROS")
axes[1].set_xlabel("Kualitas Wine")
axes[1].set_ylabel("Jumlah")

plt.tight_layout()
plt.show()
```



Gambar kiri menunjukkan kondisi awal yang tidak seimbang: Bad Wine ~984, Good Wine ~159. Gambar kanan menunjukkan hasil setelah ROS — kedua bar setara di angka ~984, distribusi sudah balance. Berbeda dengan RUS, di sini tinggi bar naik ke atas bukan turun.

5. Hasil

```
# Ringkasan Hasil
print("\nDistribusi Sebelum ROS:")
print(before_counts.rename({0: "Bad Wine", 1: "Good Wine"}))

print("\nDistribusi Sesudah ROS:")
print(after_counts.rename({0: "Bad Wine", 1: "Good Wine"}))

print(f"\nTotal data setelah ROS: {len(X_resampled)}")
```

```
Distribusi Sebelum ROS:
target
Bad Wine      984
Good Wine     159
Name: count, dtype: int64

Distribusi Sesudah ROS:
target
Bad Wine      984
Good Wine     984
Name: count, dtype: int64

Total data setelah ROS: 1968
```

Output menunjukkan Bad Wine tetap 984, Good Wine naik dari 159 menjadi 984. Total data setelah ROS menjadi 1968 — hampir dua kali lipat dari data asli (1143) karena data minoritas diperbanyak lewat duplikasi.